CAREERPILOT AI

Alexa+ Track — Complete Project, Setup & Submission Guide

A three-person team plan for an agentic career-preparation assistant

Document purpose	Recommended approach
Primary track	Alexa+ — simulated Alexa+ web experience plus a self-hosted MCP server
Project concept	Analyze a job description and resume, create a preparation plan, run quizzes, and adapt the plan based on saved results.
Build strategy	A working end-to-end MVP first; optional AWS Builder mini challenge only after the primary demo works.
Platform	Windows 11 development environment; Node.js, TypeScript, React, Vite, Express, MCP, and a lightweight database.

Prepared: 21 October 2026

Read this first: what the Alexa+ track requires

Alexa+ is Amazon's assistant; CareerPilot is the team's own application. The hackathon brief permits either (A) a self-hosted MCP server using Streamable HTTP and MCP specification 2025-11-25 or later, or (B) a simulated Alexa+ experience in a web application using an agentic tool. This plan intentionally combines a web simulator with a real MCP server so the team can demonstrate visible tool calls locally without claiming that the app is connected to Amazon's private-preview Alexa+ service.

- Do not claim that Alexa+ itself invokes the server unless you have actually completed the official partner onboarding and tested that connection.
- The official Alexa+ MCP QuickStart describes partner-preview onboarding. The hackathon FAQ says the simulated experience is allowed for participants who cannot access the preview.
- The hackathon brief states a deadline of 23 October 2026 at 12:00 PM Pacific time (24 October 2026 at 12:30 AM IST). Verify the live Devpost page for any updates.

Executive summary

CareerPilot AI is a conversational, agentic interview-preparation assistant. A user provides a resume and a job description, sets a deadline and daily study availability, and receives an evidence-backed skill-gap analysis and personalized preparation plan. The user can take a quiz; the app saves the score, identifies weaker topics, and adjusts the next plan. The simulator displays the agent's tool calls and results so judges can see that the system performs real operations instead of merely generating a chat response.

Why this is a good hackathon scope

- A clear user problem and easy-to-understand demo.
- Agentic workflow: analyze → plan → save → quiz → record score → adapt.
- MCP tools have concrete, testable functions.
- A browser-based demo can be developed on Windows without an Amazon device.
- Potential AWS Builder mini challenge via a documented AWS Bedrock integration, if credentials and time are available.

1. Problem statement and goals

Problem

Students preparing for a particular software role often use separate tools for resume review, job-description analysis, learning plans, practice questions, and progress tracking. These tools do not consistently coordinate a single plan that changes as the student improves.

Proposed solution

CareerPilot accepts job and candidate information, identifies likely skill gaps using evidence from the supplied resume, proposes a time-bounded study plan, generates quizzes, persists results, and updates the plan based on the results. The user stays in control: generated recommendations are reviewable, and any future external action (such as calendar changes) would require confirmation.

Primary demo story

1. User says/types: "I have a Java developer interview in five days. I can study two hours each day. Analyze my resume and prepare me."
2. The agent extracts job requirements and skills supported by the resume.
3. The app displays matched skills, likely gaps, and short evidence snippets or source locations.
4. The agent creates and saves a five-day plan.
5. The user takes a short quiz on one topic.

6. The app records the result and recalculates topic priority; the next plan visibly changes.
7. The simulator displays the actual tool names, arguments, results, and success/error state.

2. Scope: must-have versus stretch goals

Priority	Feature	Acceptance criteria
P0 — Must have	Job description analysis	Extract required skills, responsibilities, and seniority into validated structured data.
P0 — Must have	Resume input	Accept PDF/text or manual skills. If PDF parsing is too risky, support text paste and a sample resume first.
P0 — Must have	Skill-gap comparison	Show matched skills, gaps, and evidence; label uncertain inferences as uncertain.
P0 — Must have	Preparation plan	Generate a five-day plan based on deadline and daily available time; persist it.
P0 — Must have	Quiz + adaptive update	Generate questions, grade answers, save score, and update the next session based on results.
P0 — Must have	MCP tools	Server exposes documented tools through Streamable HTTP; tools return real structured results.
P0 — Must have	Simulator UI	Chat-like interface, plan/skill panels, tool-call trace, errors, and progress state.
P1 — Strongly recommended	Sample mode	One-click load of a clearly labelled sample resume and job description for reliable judging.
P1 — Strongly recommended	Persistence	Progress survives refresh/restart using SQLite or another configured database.
P1 — Strongly recommended	Tests + README	Fresh clone can be installed and run using documented steps.
P2 — Stretch	RAG learning resources	Search curated resources and show citations/links. Add only after P0 works.
P2 — Stretch	AWS Builder	Use and document a real AWS service integration, such as Bedrock inference, if access is ready.
P2 — Stretch	Voice input	Browser speech input only if reliable and supported; typed input is the fallback.
P2 — Stretch	Calendar integration	Do not implement unless there is time for permissions, consent, and testing.

3. Feature-by-feature functional specification

3.1 Onboarding and input

- Fields: role/job title, job description, interview date or number of preparation days, daily study minutes, and resume text/file.
- Provide a sample mode so judges can launch the workflow without entering personal data.
- Validate required fields and file types/size. Do not upload or store a resume without clear user intent.

3.2 Job requirement extraction

- Output structured skills grouped into categories such as language/framework, backend, databases, algorithms, system design, and soft skills.
- Keep the original job description available for traceability; never invent requirements not supported by the input.
- Handle malformed or incomplete model output with validation and a user-visible error.

3.3 Resume skill evidence

- Extract candidate skills and evidence snippets from the supplied resume.
- Separate explicit evidence from model-inferred skills. Do not represent inferred experience as verified fact.
- Show a gap status: evidenced match, partial/uncertain match, or not found in resume.

3.4 Planner

- Inputs: skill gaps, time available, target date, and quiz results.
- Output: day number, topic, learning objective, time estimate, practice task, and completion status.

- Keep time estimates within the user's daily availability; if impossible, explain the trade-off.

3.5 Quiz and adaptation

- Generate a small quiz (for MVP, 3–5 questions) based on a selected topic.
- Grade deterministic multiple-choice answers in application code where possible; use the model for explanations, not as the sole source of truth for the score.
- Save attempt, score, topic, timestamp, and feedback.
- Use scores and previous attempts to reorder or revise the next study session. Show the reason for the change.

3.6 Agent orchestration

- Convert user intent into a bounded sequence of available tool calls.
- Validate tool arguments, set execution limits, and handle failures.
- Do not let model output execute arbitrary shell commands, raw SQL, or arbitrary code.
- Report success only after the underlying operation succeeds. If a tool fails, explain the failure and offer a safe retry.

3.7 MCP tool trace

- Display tool name, safe input summary, status, and output summary in the simulator.
- Avoid displaying private resume content or secrets in logs.
- A demo tool trace must correspond to real server calls, not decorative hard-coded status labels.

3.8 Privacy and safety

- Use synthetic/demo candidate data for recording unless the user explicitly consents to using real data.
- Keep API keys in environment variables and out of Git.
- Do not provide employment guarantees or claim a candidate will pass an interview.
- Add a delete/reset demo-data action if persistence is enabled.

4. Architecture

Recommended architecture for the time-limited MVP:

Layer	Responsibility	Suggested implementation
Simulator frontend	Chat input, onboarding, analysis cards, plan, quiz, progress, tool trace	React + Vite + TypeScript
Agent/orchestrator	LLM call, bounded planning, tool routing, output validation	Node.js/TypeScript; provider SDK selected by team
MCP server	Expose career-preparation capabilities over Streamable HTTP	Official MCP TypeScript SDK; Express/HTTP integration
Domain/service layer	Business rules, quiz grading, plan adaptation, validation	Pure TypeScript functions
Persistence	Plans, quiz attempts, candidate/session state	SQLite for fast local MVP; MongoDB optional if team already has it
Optional AWS	Hosted model inference or other documented AWS capability	Amazon Bedrock, only after core workflow is stable

Important design decision: keep the domain functions separate from the MCP transport. The UI/agent and MCP tools should call the same validated service functions, so business logic is not duplicated and every result can be tested.

5. Proposed repository folder structure

```
careerpilot-ai/
├── README.md
├── LICENSE
├── .gitignore
```

```
├── .env.example
├── package.json                # optional npm workspaces/scripts
├── docs/
│   ├── architecture.md
│   ├── setup.md
│   ├── demo-script.md
│   ├── product-feedback.md
│   └── friction-log.md
├── samples/
│   ├── sample-resume.txt
│   └── sample-java-job-description.txt
├── apps/
│   └── simulator/
│       ├── package.json
│       ├── index.html
│       └── src/
│           ├── main.tsx
│           ├── App.tsx
│           ├── components/
│           │   ├── ChatPanel.tsx
│           │   ├── JobInputForm.tsx
│           │   ├── SkillGapPanel.tsx
│           │   ├── PlanTimeline.tsx
│           │   ├── QuizPanel.tsx
│           │   ├── ProgressPanel.tsx
│           │   └── ToolTrace.tsx
│           ├── lib/
│           │   └── api.ts
│           └── styles/
├── server/
│   ├── package.json
│   └── src/
│       ├── index.ts
│       ├── routes/
│       │   ├── analysis.ts
│       │   ├── plans.ts
│       │   └── quiz.ts
│       ├── agent/
│       │   ├── orchestrator.ts
│       │   ├── prompts.ts
│       │   └── schemas.ts
│       ├── services/
│       │   ├── jobAnalyzer.ts
│       │   ├── resumeAnalyzer.ts
│       │   ├── planner.ts
│       │   ├── quizService.ts
│       │   └── progressService.ts
│       ├── storage/
│       │   ├── db.ts
│       │   └── repository.ts
│       └── tests/
└── mcp-server/
    ├── package.json
    └── src/
```

```
        ├── index.ts             # HTTP endpoint and server startup
        ├── createServer.ts     # register tools and schemas
        ├── tools/
        │   ├── analyzeJob.ts
        │   ├── compareSkills.ts
        │   ├── createPlan.ts
        │   ├── generateQuiz.ts
        │   ├── recordQuizResult.ts
        │   └── getProgress.ts
        └── tests/
```

This is a proposed clean structure, not the exact structure of the reference repositories. A team may simplify it into one Node workspace if separate packages slow integration. Keep one shared source of truth for types/schemas where practical.

6. Dependencies and software to install

6.1 Required local software

Tool	Purpose	Action / URL
Node.js 20+	Runtime for React tooling, server, and MCP	Install LTS from https://nodejs.org/en/download; verify `node -v` and `npm -v`.
Git for Windows	Version control and GitHub workflow	https://git-scm.com/download/win; verify `git --version`.
VS Code	Editor and terminal	https://code.visualstudio.com/
GitHub account	Repository and submission	Create a team repository; agree on branch/PR rules.
LLM provider account/key	Analysis and plan generation	Choose one provider; store key only in local `.env`.
MCP Inspector	Test MCP tool discovery and calls	Use official MCP docs; run the inspector command appropriate for the selected SDK version.
Optional AWS account	AWS Builder mini challenge	Configure credentials and permissions only if using Bedrock or another AWS service.

6.2 Recommended application dependencies

Package / class	Where	Why / notes
react, react-dom	Simulator	Web UI.
vite, typescript	Simulator	Dev server/build and type checks.
zod	Server + MCP	Validate inputs and structured model output.
@modelcontextprotocol/sdk	MCP server	Use the v1 package line if following the ChartKeep sample's known 2025-11-25 implementation; pin and verify versions.
express	Server / MCP HTTP	HTTP routes and endpoint mounting if using the chosen SDK integration.
dotenv	Server	Load local environment variables in development.
LLM provider SDK	Agent	Select one provider only; don't install several before needed.
better-sqlite3 or sqlite library	Storage	Optional local persistence; choose one and test Windows install compatibility.
vitest	Tests	Optional test runner for domain functions and frontend.
@modelcontextprotocol/inspector	Developer tool	Inspect server tools and run test calls.

Version caution: MCP SDK package lines evolve. The hackathon requirement is MCP specification 2025-11-25 or later. The ChartKeep reference repository states it uses `@modelcontextprotocol/sdk` 1.32 and spec 2025-11-25. The current official TypeScript SDK documentation also describes a newer split-package v2 line. Do not mix v1 and v2 import examples. For fastest reproduction, first follow the reference repository's pinned dependency versions; upgrade only after the baseline works.

7. Repositories and documentation to refer to

Resource	URL	How to use it
Hackathon overview/resources	https://amazonappdev2026.devpost.com/resources	Confirm track choices, resources, and current instructions.
Hackathon rules	https://amazonappdev2026.devpost.com/rules	Check what must be submitted, repo access, video, feedback, and deadline.
Hackathon FAQ	https://amazonappdev2026.devpost.com/details/faqs	Confirm simulated Alexa+ path and access constraints.
Official Alexa+ MCP QuickStart	https://www.developer.amazon.com/docs/alexaplus/add-ons/mcp-toolkit-quickstart	Official protocol and onboarding requirements; private-preview access may be restricted.
ChartKeep MCP demo repo	https://github.com/sherotree/chartkeep-mcp	Primary reference for a working MCP server plus browser simulator, local setup, tool trace, tests, demo and friction log.
Smart Home Alexa+ sample	https://github.com/guptachetan1995/alexa-plus	Study human approval and safe-action UX; verify current code and license before reuse.
Northbridge Alexa MCP	https://github.com/dl88jing/northbridge-alexa-mcp	Study agent architecture, audit log, and optional Bedrock path; inspect setup before adopting.
Aawiz voice check-in	https://github.com/Aawizgroup/aawiz-voice-checkin	Study voice workflow, consent, and simulator design; verify current run instructions.
Official MCP TypeScript SDK	https://github.com/modelcontextprotocol/typescript-sdk	Use official protocol/SDK docs; choose a consistent version line.
MCP HTTP serving guide	https://github.com/modelcontextprotocol/typescript-sdk/blob/main/docs/serving/http.md	Understand Streamable HTTP server pattern; cross-check with selected SDK version.
MCP Inspector	https://github.com/modelcontextprotocol/inspector	Manually discover and call tools while developing.

Reference repo guidance: use ChartKeep as the most directly relevant runnable example. Run it unchanged first, read its README and demo script, then create a new CareerPilot repository. Do not merely rename its CSV app; replace the domain logic, implement your own workflow, tests, UI, and documentation. Preserve the reference project's license and attribution if any code is reused.

8. Step-by-step setup and development process

Phase 0 — Team setup (30–45 minutes)

8. Create a shared GitHub repository named `careerpilot-ai` and add all three teammates.
9. Agree on one package manager (npm for the MVP), one Node version, one LLM provider, and one shared sample dataset.
10. Create the folder skeleton, `.gitignore`, `.env.example`, README, and an issue/board with tasks.
11. Set up `main` as the stable branch; each person works on a feature branch and opens pull requests.
12. Assign one person as integration owner and schedule short integration checks at least twice per day.

Phase 1 — Reproduce a known-good MCP demo (45–60 minutes)

13. Clone ChartKeep into a separate learning folder; do not develop CareerPilot directly inside the reference repository.
14. Read the repository README, package scripts, server tool registration, simulator MCP client, and demo instructions.
15. Install the dependencies using the repository's own instructions; the current README states Node 20+ and `npm install`.
16. Run `npm run dev`; confirm the simulator and MCP endpoint start, the client connects, and `tools/list` / `tools/call` appear in the wire log.
17. Run `npm test` and save notes/screenshots of the successful baseline.
18. Create the new CareerPilot repo only after understanding the server/client boundary.

Phase 2 — Initialize CareerPilot (30 minutes)

```
# In a new folder
mkdir careerpilot-ai
```

```
cd careerpilot-ai
git init
npm init -y

# Create the directories described in Section 5.
# Add a .gitignore before creating .env.
```

Suggested `.gitignore` entries:

```
node_modules/
dist/
.env
.env.*
!.env.example
coverage/
*.sqlite
*.db
.DS_Store
```

Create `.env.example` with placeholder names only, e.g.:

```
PORT=8787
SIMULATOR_PORT=5173
LLM_PROVIDER=replace_me
LLM_API_KEY=
DATABASE_PATH=./data/careerpilot.sqlite
```

Never commit the real `.env` file. Do not put actual credentials in the README, screenshots, logs, or demo recording.

Phase 3 — Implement domain logic before the agent (2–3 hours)

19. Create TypeScript types/schemas for job requirements, resume evidence, skill gaps, plan days, quiz questions, and quiz attempts.
20. Implement deterministic helpers for time allocation, quiz grading, score persistence, and plan priority updates.
21. Write unit tests for these helpers using sample inputs.
22. Build the job/resume analysis function with one LLM provider and validate its JSON output.
23. Keep a fallback sample response so the team can test the UI if the provider rate-limits; label fallback/demo output honestly.

Phase 4 — Implement MCP tools (2–3 hours)

Tool	Input	Expected behavior
analyze_job_description	jobDescription	Returns structured job skills and responsibilities.
compare_resume_skills	resumeText, jobRequirements	Returns evidenced matches, uncertain matches, and gaps.
create_preparation_plan	gaps, daysAvailable, minutesPerDay	Creates and persists a time-bounded plan.
generate_quiz	topic, difficulty, questionCount	Returns validated questions and answer key for grading.
record_quiz_result	planId, topic, answers	Grades deterministically, persists attempt, returns score and weak topics.
get_learning_progress	planId or sessionId	Returns current plan, completion, scores, and updated priorities.

Implement only the first three tools initially, verify them with MCP Inspector, and then add quiz/progress tools. Every tool needs a clear description, input schema, validation, structured output, and error behavior.

Phase 5 — Build the simulator UI (2–3 hours)

24. Create a single main screen with prompt input, sample-mode button, and session settings.
25. Display matched skills/gaps, the five-day plan, quiz, and progress.
26. Add a tool trace panel showing real tool calls and their returned status.
27. Add loading states, empty states, validation messages, and retry actions.
28. Use a clean responsive design; do not spend hours on animations or unnecessary visual polish.

Phase 6 — Integrate and test (2 hours)

29. Run simulator, application API, database, and MCP server together from documented scripts.
30. Test the complete scenario using sample data from a fresh session.
31. Take a quiz, confirm the score is persisted, refresh the page, and verify the updated plan remains.
32. Test invalid inputs, model timeout/rate limit, database failure, and MCP tool error.
33. Make sure the simulator shows failures truthfully and never displays fake success.

Phase 7 — Optional AWS integration (only if MVP works)

Use Amazon Bedrock for one clearly documented model-inference call if AWS credentials, permissions, and time are ready. Keep the integration isolated behind a provider interface. Document the AWS service, the code path where it is called, setup, cost considerations, and what worked/did not work. Do not claim the AWS Builder mini challenge unless the submitted application actually uses the AWS service or qualifying Kiro Crew integration.

9. Three-person work split

Team member	Ownership	Tasks	Deliverables
Person 1 — MCP / Backend Lead	MCP server and API	Reproduce ChartKeep; implement server endpoint, tool registration, schemas, input validation, domain-service integration, errors, and MCP Inspector tests.	Working Streamable HTTP server; tool list; tool tests; API/MCP setup notes.
Person 2 — Frontend / Simulator Lead	React simulator UI	Build chat-style input, sample mode, skill-gap cards, plan timeline, quiz UI, progress panel, tool trace, responsive design, and API client.	Working web simulator; loading/error states; demo-ready UI.
Person 3 — Agent / Data / QA Lead	LLM workflow, persistence, tests and submission	Implement prompts/output schemas, job/resume analysis, planner logic, quiz adaptation, persistence integration, sample data, test scenarios, README/demo script/product feedback.	End-to-end agent workflow; persistence tests; README; demo video and submission checklist.

Shared responsibilities and integration contract

- All three members review the tool schemas and agree on field names before coding in parallel.
- Person 1 owns MCP tool contracts; Person 2 owns UI components; Person 3 owns agent/business logic and end-to-end QA.
- No one merges a breaking change without notifying the other two.
- Integrate early: after the first tool is callable, connect it to the UI immediately.
- All members contribute to the final demo, but one person should operate the live application while another narrates.

Suggested branches

```
main
feature/mcp-tools
feature/simulator-ui
feature/agent-and-tests
```

Suggested integration milestones

Checkpoint	Required proof
Checkpoint 1	Reference MCP demo runs on each developer's machine or at least on the integration owner's machine.
Checkpoint 2	CareerPilot's first MCP tool returns a real result in MCP Inspector.
Checkpoint 3	Simulator invokes the tool and renders the result.
Checkpoint 4	Quiz result persists and changes the plan.
Checkpoint 5	Fresh-clone setup and three-minute recording work.

10. Environment configuration and run commands

Exact scripts depend on the final package layout and SDK version. The team must standardize them in the root `package.json`; avoid undocumented commands that only work on one person's machine.

Recommended final scripts (names may be adapted):

```
npm run dev          # start simulator + API + MCP server
npm run dev:web      # simulator only
npm run dev:server   # API / agent only
npm run dev:mcp      # MCP server only
npm test             # unit and integration tests
npm run typecheck    # TypeScript checks
npm run build        # production build
```

If using the ChartKeep baseline as a guide, its README documents:

```
npm install
npm run dev
npm test
```

ChartKeep's documented local endpoints are simulator `http://127.0.0.1:5173`, MCP endpoint `http://127.0.0.1:8787/mcp`, and health check `http://127.0.0.1:8787/health`. These endpoints belong to ChartKeep's sample and are not automatically CareerPilot endpoints.

11. Test plan and definition of done

Test	Expected result
Happy path	Sample resume + job description produce skill gaps and a saved plan.
Input validation	Empty job description or malformed request returns a useful validation message.
Evidence quality	No unsupported claim is presented as a verified resume fact.
Plan constraints	Daily plan does not exceed the user's time budget without explanation.
Quiz grading	Known answer set produces deterministic score.
Adaptation	A weak quiz result changes topic priority in the next plan.
Persistence	Refresh/restart retains plan and quiz attempt.
MCP protocol	Client can initialize, list tools, and call each tool over Streamable HTTP.
Failure behavior	LLM timeout/tool/database error is visible and does not fabricate success.
Security	No API key or real resume appears in Git or public logs.
Fresh setup	A teammate can clone and run from README instructions.

12. GitHub, license, and collaboration

34. Create the repository under one team member's GitHub account or an organization and add the other two members.
35. Choose a public repository with an open-source license or keep it private and share it with the required hackathon accounts, as the official submission rules specify.
36. Commit `.env.example`, not `.env`; use synthetic sample data.
37. Include source code, assets, setup steps, tests, license, architecture diagram or description, and demo script.
38. If reusing code from reference repositories, review the LICENSE, retain required notices, and describe what was independently created or changed.

13. Submission deliverables

Deliverable	What to prepare
Project description	Problem, target user, core workflow, how agent/MCP works, novelty, limitations, and impact.

Repository	All source, assets, setup instructions, license or private sharing configured, no secrets.
Demo video	Public YouTube/Vimeo, English, under 3 minutes, shows actual running workflow and MCP/simulation.
Product feedback	For every tool/API/SDK: what it was used for, what worked, what needs improvement, onboarding experience, would build again.
Track selection	Alexa+ primary track; AWS Builder only if qualifying AWS integration is implemented.
Open Source mini challenge	Separate additional project/contribution and required URL/description if entering.
Friction log	Actual task, steps, expected vs actual, severity, workaround, actionable improvement; optional bonus opportunity.
Feature requests	Optional request, why it matters, urgency.

Three-minute demo script

Time	Scene
0:00–0:20	State the problem and show CareerPilot.
0:20–0:45	Load sample job/resume and submit request.
0:45–1:15	Show skill gaps and explain evidence.
1:15–1:45	Show saved five-day plan.
1:45–2:15	Complete quiz; show persisted score and plan adaptation.
2:15–2:40	Show real MCP tool call trace and result.
2:40–3:00	Summarize differentiation, stack, and impact.

14. Risks and mitigations

Risk	Mitigation
Alexa+ private preview unavailable	Use the allowed web simulator and clearly describe it as simulated; do not claim official Alexa+ connectivity.
SDK/spec version confusion	Pin one tested SDK version and use docs for that version; verify Streamable HTTP and required spec version.
LLM API quota/rate limits	Keep a sample mode, use short prompts, handle errors, and never fake live success.
Too much scope	Ship analysis → plan → quiz → adaptation; defer RAG, voice, calendar, and complex auth.
Team integration conflicts	Agree on schemas early; use feature branches; integrate the first tool into UI immediately.
Sensitive resume data	Use synthetic data in demo, avoid storing more than needed, keep secrets out of Git.
No public hosting time	Local demo is better than broken hosting if allowed by submission path; ensure the video is clear and reproducible.

15. Recommended execution schedule

The deadline in the supplied hackathon brief is very close. Use this as a compressed plan, not a reason to skip testing. Verify the current Devpost page before relying on the deadline.

Window	Focus	Exit criteria
Day 1 — first half	Install, run reference repo, create CareerPilot repo and contracts	Known-good reference; server skeleton; agreed schemas.
Day 1 — second half	Implement first tools, UI skeleton, LLM analysis	One real tool call displayed in UI.
Day 2 — first half	Plan persistence, quiz, adaptation, tool trace	Full end-to-end happy path.
Day 2 — second half	Test, polish, README, friction log, recording	Fresh-clone run and under-3-minute demo ready.
Before deadline	Submit all fields and verify links/access	Submission confirmation; backup video exists.

16. Official links and source notes

Hackathon resources: https://amazonappdev2026.devpost.com/resources

Hackathon rules: https://amazonappdev2026.devpost.com/rules

Hackathon FAQ: https://amazonappdev2026.devpost.com/details/faqs

Alexa+ MCP QuickStart: https://www.developer.amazon.com/docs/alexaplus/add-ons/mcp-toolkit-quickstart

ChartKeep MCP (recommended reference): https://github.com/sherotree/chartkeep-mcp

Smart Home Alexa+ demo: https://github.com/guptachetan1995/alexa-plus

Northbridge Alexa MCP: https://github.com/dl88jing/northbridge-alexa-mcp

Aawiz Voice Check-in: https://github.com/Aawizgroup/aawiz-voice-checkin

Official MCP TypeScript SDK: https://github.com/modelcontextprotocol/typescript-sdk

MCP TypeScript SDK HTTP serving docs: https://github.com/modelcontextprotocol/typescript-sdk/blob/main/docs/serving/http.md

MCP Inspector: https://github.com/modelcontextprotocol/inspector

Node.js downloads: https://nodejs.org/en/download

Git for Windows: https://git-scm.com/download/win

VS Code: https://code.visualstudio.com/

Source-grounding note: the track requirements and submission deliverables in this guide are based on the hackathon brief supplied in the conversation and the linked official pages. The project idea, feature scope, folder structure, team split, and schedule are recommendations for this team, not requirements issued by Amazon. Public reference repositories can change; verify current README instructions, dependency versions, license, and commit state before copying code.

Appendix A — Quick-start checklist

- ☐All teammates can run `node -v`, `npm -v`, and `git --version`.
- ☐ChartKeep reference runs locally and team understands its MCP client/server flow.
- ☐CareerPilot repository has a README, license choice, .gitignore, and .env.example.
- ☐The first MCP tool is discoverable and callable through Inspector.
- ☐The simulator shows a real tool call and result.
- ☐Resume/job analysis, plan, quiz, and adaptation work in sample mode.
- ☐Progress persists and tests pass.
- ☐No secrets or private user data are committed.
- ☐Demo video is public, English, under three minutes.
- ☐Submission fields, track selections, repo access, and feedback are complete.